# Trabajo Práctico Obligatorio

## Miti: aplicación móvil para coordinar el hogar compartido

*Etapa 1 — Preentrega de análisis y diseño*

*"La casa, miti y miti"*

*Universidad Argentina de la Empresa*
*Facultad de Ingeniería y Cs. Exactas*
*Desarrollo de Aplicaciones I*

| | |
|---|---|
| Universidad | UADE · Facultad de Ingeniería y Ciencias Exactas |
| Materia | Desarrollo de Aplicaciones I |
| Trabajo | TPO 2026 · Etapa 1: preentrega de análisis y diseño |
| Aplicación | Miti · "La casa, miti y miti" |
| Integrantes | Rocío Bottinelli · Juan Cruz Bautista Serra |
| Repositorio | github.com/rociobottinelli/miti |
| Diseño en Figma | miti-UI en Figma |

# *Indice*

**Resumen** 4
**1. Nombre provisorio** 4
**2. Descripción del problema** 4
Evidencia: entrevista inicial 4
**3. Usuarios principales** 5
Usuario principal: quien coordina el hogar 5
Usuario secundario: la otra persona del hogar 5
**4. Contexto de uso** 5
Mirada de computación ubicua 6
**5. Propuesta de solución** 6
Escenario principal de uso 7
**6. Por qué una aplicación móvil** 7
**7. Caso de negocio y propuesta de valor** 7
Diferencia con las alternativas existentes 8
**8. Alcance y fuera de alcance** 8
Incluido en la primera versión 8
Fuera de alcance 8
**9. Requisitos funcionales** 9
**10. Requisitos no funcionales** 10
**11. Diseño en Figma** 11
Pantallas principales 11
Estados a diseñar 12
**12. Flujo de pantallas** 12
Flujo 1: primer uso 13
Flujo 2: rutinas y tareas 13
Flujo 3: compra, gasto y pago (caso de uso principal) 14
**13. Arquitectura** 14
Qué le exige el problema a la arquitectura 14
Flujo de datos 15
Topología de infraestructura 16
Decisiones de arquitectura y su justificación 16
D01 · MVVM en la capa de presentación 16
D02 · Tres capas con un dominio en Kotlin puro 16
D03 · Los mismos casos de uso para la app, el widget y la notificación 17
D04 · Room como única fuente de verdad y flujo de datos unidireccional 17
D05 · Repositorios definidos como interfaces en el dominio 17
D06 · Sincronización con WorkManager y cambios pendientes marcados en Room 17
D07 · Identificadores UUID generados en el dispositivo y borrado lógico 18
D08 · Conflictos: marcas que solo se agregan y "gana la última modificación" para el resto 18
D09 · Los datos derivados se calculan, no se guardan 18
D10 · Avisos con alarmas locales y no con push desde un servidor 18
D11 · Un backend central; los celulares nunca se comunican entre sí 19
Trazabilidad entre requisitos y decisiones 19
**14. Persistencia y estrategia Offline First** 19
Entidades principales 19

Comportamiento según la conectividad..... 20
Fuente remota y actualización..... 20
Errores y conflictos de sincronización..... 21
**15. Tecnologías previstas..... 21**
**16. Repositorio y forma de trabajo..... 22**
**17. Integrantes, roles y responsabilidades..... 22**

# Resumen

Miti es una aplicación Android pensada para que dos personas que conviven coordinen en un solo lugar sus rutinas, tareas, compras y gastos compartidos, sin que la organización dependa de que una sola de ellas recuerde y controle todo.

**Problema → Usuario → Solución → Valor**

- **Problema:** actualmente, la coordinación del hogar está fragmentada entre WhatsApp, notas, la puerta de la heladera y un Excel, y recae en una sola persona.
- **Usuario:** quien hoy organiza el hogar, y la persona con quien convive.
- **Solución:** un widget con las rutinas del día que cualquiera marca en un toque, tareas programadas que avisan a su responsable, y una lista de compras conectada a un registro de gastos con un balance siempre calculado.
- **Valor:** una menor carga de organización, facilitar el reparto más equitativo y la confianza en los números, sin consolidaciones manuales a fin de mes.

# 1. Nombre provisorio

Elegimos el nombre **Miti**, de la expresión "miti y miti" por el reparto 50/50 de gastos y de carga mental de las tareas que la aplicación busca.

# 2. Descripción del problema

En un hogar compartido, las tareas, compras y gastos están relacionados entre sí (una compra genera un gasto), pero se gestionan por canales separados. Nadie puede visualizar rápidamente qué es lo que falta, quién se tiene que ocupar de qué y cómo se repartieron los gastos que se realizan. Como consecuencia, una persona concentra la carga cognitiva de recordar, pedir y controlar, y la otra depende de que le avisen.

| Pregunta | Respuesta |
|---|---|
| ¿Qué problema existe? | La coordinación del hogar está fragmentada y no hay una visión común del estado de las responsabilidades compartidas. |
| ¿Quién lo experimenta? | Dos personas que conviven, en especial quien se encarga de organizar. La primera versión se enfoca en parejas o convivientes de a dos. |
| ¿En qué contexto ocurre? | En el día a día (supermercado, transporte, trabajo, casa) y a fin de mes, al repartir gastos. |
| ¿Cómo se resuelve hoy? | Grupos de WhatsApp, notas del celular, un calendario en la heladera, planillas de Excel y la memoria de quien organiza. |
| ¿Qué dificultades presenta? | Olvidos, compras duplicadas, consolidación manual y tediosa de gastos y cuotas, errores que nadie detecta y carga desigual que afecta el vínculo. |
| ¿Por qué una app móvil? | La coordinación ocurre en movimiento, en momentos breves y con una conectividad variable (ver sección 6). |

## Evidencia: Entrevista inicial

Se entrevistó a una persona que convive en pareja y concentra la organización del hogar. Maneja todo desde las notas del celular, deja a su pareja una lista de las tareas y fechas importantes en un calendario de la heladera, usa un grupo de WhatsApp para la lista del supermercado y consolida los gastos en un Excel a fin de mes, incluyendo las cuotas. Su pareja declara depender completamente de ella.

| Pregunta | Respuesta | Implicancia para el diseño |
|---|---|---|
| ¿Qué es lo que más te molesta de organizar todo? | Consolidar todo a fin de mes. | El balance debe estar siempre calculado, sin paso de consolidación. |
| ¿Qué salió mal por no tenerlo registrado? | No lo sabe, ni se enteraría. | El sistema actual no es verificable: la app aporta trazabilidad de cada gasto. |
| ¿Cómo pasás los gastos al Excel? | Gasto por gasto, al terminar el mes, para ver quién le debe a quién. | El objetivo del eje gastos es un único dato: el balance entre las dos personas. |
| ¿Qué datos anotás de cada gasto? | Monto, cuotas faltantes, quién pagó y para quién era (50/50 o 100/0). | Define el modelo de gasto: dos modos de reparto y cuotas simples. |
| ¿Qué te gustaría que haga tu pareja sin pedírselo? | Todo lo que se pueda. | Confirma el deseo de redistribuir la carga. Hay que repreguntar con casos concretos. |
| ¿Probaste alguna app? | No. Quiere algo que se adapte a su forma de organizarse. | Las apps genéricas no calzan con su flujo. Riesgo: diseñar para una sola persona. |

**Hallazgo de diseño:** el calendario de la heladera funciona porque está siempre a la vista y consultarlo no requiere esfuerzo. La app debe acercarse a esa experiencia: mostrar las rutinas del día en un widget de la pantalla de inicio, marcarlas sin abrir la app y avisar sin que haya que entrar a revisar constantemente..

# 3. Usuarios principales

## Usuario principal: quien coordina el hogar

| Aspecto | Descripción |
|---|---|
| Características | Adulto de 25 a 50 años que convive con su pareja u otra persona. Usa el celular para todo, pero no usa herramientas de productividad. Trabaja y organiza la casa en huecos del día. |
| Necesidad principal | Dejar de ser la única persona que sabe qué falta, qué hay que hacer y quién pagó qué. |
| Frecuencia | Varias veces por día, en interacciones de menos de un minuto. |
| Restricciones | Poco tiempo, atención dividida, uso con una mano. Si registrar algo lleva más que un mensaje de WhatsApp, que es lo que actualmente hace, abandona la app. |
| Dificultades actuales | Información repartida entre WhatsApp, notas, heladera, Excel y memoria. Consolidación manual de gastos a fin de mes revisando varias fuentes. |

## Usuario secundario: la otra persona del hogar

Consulta qué le toca, marca cosas como hechas y carga gastos. Su necesidad es no tener que preguntar ni esperar a que le avisen. Su adopción es el principal riesgo del producto: si no usa la app, la carga vuelve al coordinador. Por eso su experiencia debe ser todavía más simple: ver lo suyo y marcarlo en un toque.

# 4. Contexto de uso

La app se usa en momentos cortos y fragmentados, casi siempre haciendo otra cosa.

| Momento | Situación | Conectividad | Qué necesita de la app |
|---|---|---|---|
| Mañana, en casa | Planifica el día, reparte tareas. | Buena (Wi-Fi) | Cargar varios ítems rápido y asignarlos. |
| Transporte | Se acuerda de algo que falta. | Intermitente | Registrar en segundos, con una mano, sin depender de la señal. |

| Momento | Situación | Conectividad | Qué necesita de la app |
|---|---|---|---|
| Supermercado | Carrito en una mano, ruido, apuro. | Mala o nula | Ver la lista sin conexión y tachar ítems con un toque. |
| Trabajo | Recibe una tarea asignada. | Buena | Enterarse sin abrir la app (notificación). |
| Fin de mes | Quiere saber quién le debe a quién. | Buena | Ver el balance ya calculado, con las cuotas del mes consideradas y detaladas. |

### Mirada de computación ubicua

Siguiendo a Mark Weiser, la app debe aparecer en el momento justo y desaparecer después:

- Las rutinas del día están a la vista en un widget, como el calendario de la heladera, y se marcan con un toque sin abrir la app.
- Registrar algo lleva como máximo 3 toques desde la pantalla principal.
- La lista de compras está disponible sin conexión en el supermercado.
- Las tareas programadas avisan a su responsable a la hora indicada para que nadie tenga que recordarlas.
- El balance no necesita ser "armado" por una persona: ya está calculado cuando se lo consulta a fin de mes, o en el momento que lo necesiten.

## 5. Propuesta de solución

Miti es una app compartida por dos personas con cuatro tipos de ítem. Las **rutinas** se marcan desde un widget, las **tareas programadas** avisan a su responsable, las **compras** se tachan en el supermercado y los **gastos** alimentan un balance que se recalcula al instante. Al terminar una compra, la app ofrece registrar el gasto asociado.

| Tipo | Ejemplo | Responsable | Repetición | Aviso | Al completarse |
|---|---|---|---|---|---|
| Rutina | Darle de comer a Pancho, el perro, 2 veces por día | Nadie fijo: la marca cualquiera | Todos los días o ciertos días de la semana, de 1 a 3 veces por día | No; se ve en el widget | Registra quién y cuándo ("Tomás · 8:15"); se reinicia al día siguiente |
| Tarea programada | Pagar el ABL el jueves a las 18 | Una persona | Una sola vez, con fecha y hora | Sí, al responsable, a la hora fijada | Pasa a resuelta |
| Compra | Leche | — | — | No | Pasa al carrito; al terminar la compra se ofrece registrar el gasto |
| Gasto | Supermercado $ 24.000, a medias | Quien pagó | Opcionalmente en cuotas iguales | No | Actualiza el balance |

| Elemento | Definición |
|---|---|
| Nombre provisorio | Miti |
| Usuario principal | Quien coordina el hogar |
| Propuesta de valor | Las obligaciones del hogar a la vista y las cuentas siempre al día, sin que una persona tenga que recordar, pedir ni consolidar. |
| Beneficio esperado | Menos olvidos y duplicaciones, carga de organización mejor repartida y cero trabajo de consolidación a fin de mes. |

### Escenario principal de uso

1. A las 8, Tomás le da de comer a Pancho y toca el check en el widget, sin abrir la app. Ana ve en su widget "1 de 2 · Tomás 8:15" y no lo repite.
2. Ana agrega "leche" y "detergente" a compras y crea la tarea "Pagar ABL" para Tomás, el jueves a las 18, con aviso.
3. A la tarde, Tomás pasa por el supermercado. Adentro no tiene señal, pero la lista está disponible porque se sincronizó antes. Tacha los productos y, al terminar, registra el gasto: pagó él, a medias.
4. Al recuperar la conexión, los cambios se sincronizan. Ana ve la compra resuelta y el balance actualizado sin haber preguntado nada.
5. El jueves a las 18, a Tomás le llega la notificación "Pagar ABL" y la marca como hecha desde la misma notificación.
6. A fin de mes, en lugar de armar el Excel, Ana abre el balance y ve quién le debe a quién, incluidas las cuotas del mes. Quien debe transfiere el dinero, lo registra como pago en la app y el balance queda en "Están a mano".

## 6. Por qué una aplicación móvil

La coordinación del hogar ocurre en movimiento y en el momento en que surge la necesidad, no sentado frente a una computadora. Una web o un sistema de escritorio no llega a los momentos cotidianos que esta aplicación necesita impactar para solucionar los puntos de dolor del usuario.

| Capacidad móvil | Qué problema resuelve |
|---|---|
| Widget de pantalla de inicio | Replica el calendario de la heladera: las rutinas del día a la vista y marcables sin abrir nada. |
| Notificaciones locales programadas | La tarea avisa a su responsable a la hora justa, en reemplazo de "te lo recuerdo por WhatsApp". Funcionan aunque no haya conexión. |
| Almacenamiento local y funcionamiento offline | La lista de compras se usa en supermercados con mala señal; registrar o marcar algo no puede depender de la conexión. |
| Sincronización en segundo plano | Los cambios hechos sin conexión se envían solos al volver la señal, sin acción del usuario. |
| Acceso inmediato y uso con una mano | Registrar algo en segundos, en el colectivo o con el carrito, antes de olvidarlo. |

No se requiere utilizar ubicación, cámara ni sensores porque no resuelven ningún problema identificado. Se reconsiderarían ante una necesidad concreta, como fotografiar tickets o comprobantes.

## 7. Caso de negocio y propuesta de valor

Su valor principal es social y operativo: redistribuir la carga de organización del hogar y eliminar el trabajo manual de consolidar gastos.

| Tipo de valor | Cómo se genera |
|---|---|
| Social | La organización deja de depender de una sola persona. Cada una ve y resuelve lo que le toca sin que se lo pidan. |
| Operativo | El balance se calcula al registrar cada gasto; desaparece la consolidación mensual en Excel. |
| Confiabilidad | Cada gasto queda registrado en el momento, con quién lo cargó, visible para ambos. Los errores dejan de ser invisibles. |

| Tipo de valor | Cómo se genera |
|---|---|
| Experiencia | Una sola app reemplaza WhatsApp, notas, heladera y planilla. |

### Diferencia con las alternativas existentes

Splitwise resuelve gastos; Cozi, OurHome o FamilyWall resuelven organización familiar; las notas compartidas resuelven listas. Miti se diferencia por conectar los ejes (una compra resuelta genera su gasto) y por el manejo simple de cuotas compartidas, que la usuaria entrevistada hoy lleva a mano.

## 8. Alcance y fuera de alcance

La primera versión cubre rutinas con widget, tareas programadas con aviso, compras y gastos con balance, para un hogar de dos personas, con y sin conexión.

### Incluido en la primera versión

- Crear cuenta, iniciar y cerrar sesión con email y contraseña.
- Onboarding de 3 pantallas en el primer uso, que se puede omitir desde cualquiera de ellas.
- Crear un hogar y unirse con un código de 6 caracteres. Máximo dos integrantes.
- **Rutinas:** todos los días o ciertos días de la semana, de 1 a 3 veces por día, sin responsable fijo. Cualquiera las marca; queda registrado quién y cuándo, y se reinician cada día.
- **Tareas programadas:** fecha, hora, responsable y aviso opcional.
- **Compras:** lista compartida; al terminar la compra, la app ofrece registrar el gasto.
- **Gastos:** monto, quién pagó, reparto a medias (50/50) o de uno solo (100/0), y cuotas iguales (cantidad total y cuota actual).
- **Balance** mensual entre las dos personas, siempre actualizado, incluyendo la cuota del mes. Se puede registrar un pago para saldar la deuda, total o parcial; lo que no se salda pasa como saldo pendiente al mes siguiente.
- **Widget** de pantalla de inicio en dos tamaños (4x2 y 2x2) con las rutinas de hoy, marcables desde el widget.
- **Notificaciones locales:** a la hora de una tarea programada (al responsable) y cuando la sincronización trae una tarea nueva asignada.
- Funcionamiento offline con sincronización automática al recuperar conexión.

### Fuera de alcance

| Funcionalidad | Motivo |
|---|---|
| Hogares de más de dos personas | Complica el balance (deudas cruzadas). Se evaluará en una versión futura. |
| Recurrencias complejas en rutinas (cada 15 días, mensual, cada X horas) | Los casos relevados son diarios o semanales. |
| Tareas programadas que se repiten (ej. pagar el ABL todos los meses) | Por ahora se crean de nuevo; se evaluará después de validar las rutinas. |
| Recordatorios de rutinas y rotación automática de responsables | Las rutinas se resuelven a la vista en el widget, sin avisos. |
| Notificaciones push desde servidor | Requieren backend propio (Cloud Functions/FCM). Se usan notificaciones locales. |
| Recuperación de contraseña e ingreso con Google | No son parte del problema; se agregan si el tiempo lo permite. |

| Funcionalidad | Motivo |
|---|---|
| Vista de calendario | La lista de próximas tareas cubre la necesidad inicial. |
| Categorías, precios o historial de productos | No aparecen en la entrevista. |
| Repartos en porcentajes arbitrarios | La entrevista solo usa 50/50 y 100/0. |
| Intereses, cierre de tarjeta o cuotas de montos distintos | Se modelan cuotas iguales, como en el Excel actual. |
| Pagos reales desde la app (Mercado Pago, transferencias) | Miti registra que se pagó; el pago en sí se hace por fuera de la app. |
| Integración con Google Calendar, chat y lectura de tickets | Sin evidencia de necesidad en esta etapa. |

# 9. Requisitos funcionales

Los cuatro requisitos cubren el flujo principal de punta a punta: registrar, consultar, completar y ver el balance. El registro de usuario y el onboarding son soporte del flujo, no requisitos principales.

| ID | Requisito | Descripción | Usuario | Criterio de aceptación |
|---|---|---|---|---|
| RF01 | Registrar un ítem del hogar | El usuario crea una rutina (frecuencia y veces por día), una tarea programada (responsable, fecha, hora y aviso) o una compra. | Cualquier integrante | Al confirmar, el ítem aparece de inmediato, aun sin conexión, y en el otro dispositivo al sincronizar. Una tarea con aviso genera una notificación al responsable a la hora fijada, aunque no haya conexión. |
| RF02 | Consultar lo de hoy | El usuario ve en la app y en el widget las rutinas del día con su progreso y quién las marcó, y en la app las próximas tareas, filtrables por responsable. | Cualquier integrante | Sin conexión, la app y el widget muestran los últimos datos guardados, y la app indica la hora de la última sincronización. |
| RF03 | Completar un ítem | El usuario marca una rutina (desde la app o el widget), una tarea (desde la app o la notificación) o una compra. Al terminar una compra, la app ofrece registrar el gasto. | Cualquier integrante | Marcar una rutina desde el widget no requiere abrir la app, registra quién y a qué hora, y se refleja en la app. Al día siguiente la rutina vuelve a figurar pendiente. Al aceptar el gasto de una compra, se crea un gasto vinculado. |
| RF04 | Registrar gastos y pagos, y consultar el balance | El usuario registra un gasto (monto, quién pagó, reparto 50/50 o 100/0, cuotas) y ve cuánto le debe una persona a la otra en el mes. También registra un pago para saldar la deuda, total o parcial. | Cualquier integrante | Al guardar o editar un gasto, el balance se actualiza sin otra acción. Un gasto en cuotas suma solo la cuota correspondiente al mes consultado. Un pago registrado se descuenta del balance; si al cerrar el mes queda deuda, aparece como saldo pendiente en el mes siguiente. |

**Regla de cálculo del balance (ejemplo):** Tomás paga $ 30.000 en 3 cuotas, a medias. Cada cuota vale $ 10.000 y a Ana le corresponde la mitad, así que en cada uno de los tres meses el balance suma "Ana le debe $ 5.000 a Tomás". Si el primer mes Ana registra un pago de $ 5.000, ese mes queda "Están a mano"; si no paga, el mes siguiente arranca con $ 5.000 de saldo pendiente más la nueva cuota.

# 10. Requisitos no funcionales

| ID | Categoría | Requisito | Cómo se verifica |
|---|---|---|---|
| RNF01 | Comportamiento offline | Registrar, consultar y completar ítems, marcar rutinas desde el widget y registrar gastos funcionan sin conexión con los datos guardados localmente. | Modo avión: se crea una tarea, se marca una rutina desde el widget y se carga un gasto; se reinicia el teléfono y siguen visibles. |
| RNF02 | Confiabilidad | Ningún cambio hecho sin conexión se pierde: se envía automáticamente dentro del primer minuto con conexión. | Se hacen 3 cambios en modo avión, se reactiva la red y aparecen en el otro dispositivo. |
| RNF03 | Usabilidad | Marcar una rutina desde el widget requiere 1 toque; registrar un ítem desde la app, como máximo 3 toques más el texto. | Conteo de toques sobre el flujo real. |
| RNF04 | Rendimiento | La pantalla Hoy muestra datos locales en menos de 1 segundo desde que se abre la app, con o sin conexión. | Medición en un dispositivo de gama media con 200 ítems cargados. |
| RNF05 | Consistencia widget–app | Una rutina marcada en la app se ve marcada en el widget en menos de 5 segundos, y viceversa al abrir la app. | Prueba cruzada en un mismo dispositivo. |
| RNF06 | Puntualidad de avisos | La notificación de una tarea programada llega con un desvío máximo de 5 minutos respecto de la hora fijada, con o sin conexión, y se mantiene después de reiniciar el teléfono. | Tarea programada a 10 minutos, en modo avión y con un reinicio intermedio. |
| RNF07 | Usabilidad (una mano) | Las acciones principales (agregar, marcar) están en la mitad inferior de la pantalla, con áreas táctiles de al menos 48 × 48 dp. El onboarding se puede omitir desde cualquiera de sus pantallas y no vuelve a mostrarse. | Revisión del diseño y del Layout Inspector. |
| RNF08 | Accesibilidad | Todos los íconos de acción tienen descripción para TalkBack, el texto respeta el tamaño de fuente del sistema hasta 200% y la diferencia entre rutina y tarea no depende solo del color. | Recorrido del flujo principal con TalkBack activado y fuente al máximo. |
| RNF09 | Recuperación ante errores | Si un envío falla, la app reintenta automáticamente y muestra un indicador de "pendiente de sincronizar" en el ítem, sin bloquear el uso. | Se corta la red durante una sincronización y se verifica el reintento. |
| RNF10 | Privacidad y seguridad | Cada hogar solo puede leer y escribir sus propios datos; el acceso remoto requiere sesión autenticada y las contraseñas nunca se guardan en el dispositivo. | Intento de lectura de un hogar ajeno desde otra cuenta: debe ser rechazado. |
| RNF11 | Consistencia de datos | El balance mostrado coincide con el cálculo manual de los gastos registrados. | Tests unitarios del caso de uso de balance con casos 50/50, 100/0 y en cuotas. |
| RNF12 | Compatibilidad | La app funciona en Android 8.0 (API 26) o superior. | Ejecución en emulador API 26 y en un dispositivo actual. |
| RNF13 | Mantenibilidad | La lógica de negocio (balance, cuotas, reinicio diario de rutinas, sincronización) vive en la capa de dominio, sin dependencias de Android. | Revisión de dependencias y tests unitarios sin emulador. |

# 11. Diseño en Figma

El diseño sigue Material Design 3, el sistema de diseño de Android, y muestra cómo una persona recorre la app para resolver el caso principal. Enlace: [miti-UI en Figma].

## Pantallas principales

| Pantalla | Propósito | Componentes principales | Acciones |
|---|---|---|---|
| Splash | Mostrar la marca mientras la app carga. | Ícono de Miti centrado sobre fondo de color,. | Ninguna; pasa sola al onboarding o a Hoy. |
| Onboarding (3 pantallas) | Explicar rutinas, tareas programadas y cuentas en el primer uso. | Ilustración, título, indicador de página. | Siguiente, Omitir. |
| Crear cuenta / Iniciar sesión | Identificar a cada integrante. | Campos de nombre, email y contraseña. | Crear cuenta, ingresar, alternar entre ambas. |
| Tu hogar | Formar el hogar de dos. | Cards "Crear un hogar" (con código para compartir) y "Unirme con un código". | Crear, copiar o compartir código, unirse. |
| Hoy | Ver qué hay que hacer hoy y a quién le toca. | Secciones "Rutinas de hoy" y "Próximas tareas", chips de filtro, banner de conexión, botón flotante "+". | Marcar, filtrar, abrir detalle, agregar. |
| Agregar (hoja inferior) | Elegir qué registrar. | Opciones Rutina, Tarea, Compra, Gasto. | Elegir tipo. |
| Formularios de rutina y tarea | Registrar en segundos. | Rutina: nombre, frecuencia, veces por día. Tarea: nombre, responsable, fecha, hora, aviso. | Guardar, cancelar. |
| Compras | Lista del supermercado. | Lista con checkbox, sección "En el carrito", campo fijo para agregar ítems. | Agregar, tachar, terminar la compra. |
| Registrar gasto | Cargar un gasto, suelto o desde una compra. | Monto, descripción, quién pagó, reparto, cuotas. | Guardar, cancelar. |
| Balance | Saber quién le debe a quién este mes. | Selector de mes, resultado destacado con botón "Registrar pago", lista de gastos y pagos del mes, saldo pendiente del mes anterior. | Cambiar mes, abrir gasto, agregar gasto, registrar pago. |
| Ajustes | Administrar el hogar. | Integrantes, código del hogar, notificaciones, cerrar sesión. | Compartir código, cerrar sesión. |
| Widgets 4x2 y 2x2 | Rutinas del día a la vista. | Lista de rutinas con check y progreso; versión chica con la próxima rutina. | Marcar hecha, abrir la app. |
| Notificación de tarea | Avisar a la hora fijada. | Notificación Android con acción. | Marcar como hecha, abrir. |

La navegación principal es una barra inferior con tres destinos: **Hoy**, **Compras** y **Balance**. Ajustes se abre desde el avatar del hogar.

### Estados a diseñar

| Pantalla | Carga | Contenido | Vacío | Error | Offline |
|---|---|---|---|---|---|
| Hoy | Esqueleto de lista (solo la primera vez, sin datos locales). | Rutinas y tareas. | "Agregá tu primera rutina"; y "Todo al día por hoy" cuando está todo hecho. | Aviso de sincronización fallida, sin ocultar la lista. | Banner "Sin conexión · última sincronización hh:mm" y la lista local. |
| Compras | Esqueleto. | Lista y carrito. | "No falta nada". | Igual que Hoy. | Lista local y banner. |
| Balance | Esqueleto del resultado. | Resultado y gastos. | "Todavía no hay gastos este mes"; "Están a mano" con saldo cero. | Igual que Hoy. | Balance con datos locales y banner. |
| Widget | — | Rutinas de hoy. | "Todavía no hay rutinas". | — | Funciona igual; marca en local. |
| Crear cuenta, ingreso y hogar | Indicador en el botón. | Formulario. | — | "Email o contraseña incorrectos", "Código inválido", "El hogar ya tiene dos integrantes". | "Necesitás conexión para ingresar". |
| Formularios de ítem y gasto | — | Formulario. | — | Validación en línea (monto vacío, cuota mayor al total). | Se guarda igual; el ítem muestra "pendiente de sincronizar". |

También se diseña el estado **permiso de notificaciones rechazado**: un aviso en Hoy que explica que no llegarán los avisos de tareas, con acceso a la configuración.

## 12. Flujo de pantallas

Se documentan tres flujos: el primer uso, las rutinas y tareas, y el caso de uso principal (compra, gasto, balance y pago). Cada uno indica el punto de entrada, las pantallas, las decisiones, las operaciones críticas y los finales posibles. La navegación principal es una barra inferior con Hoy, Compras y Balance; Ajustes se abre desde el avatar del hogar.

## Flujo 1: primer uso

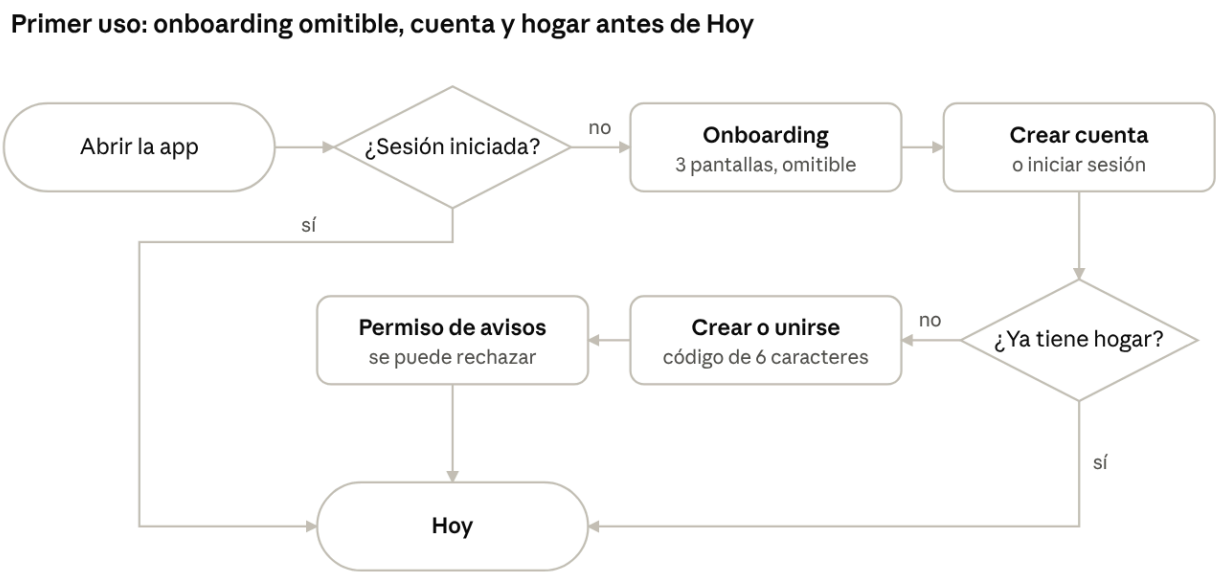



*Figura: Flujo 1 · primer uso*

El onboarding y el pedido de cuenta y hogar aparecen solo la primera vez; si ya hay sesión, la app abre directo en Hoy. Crear cuenta y unirse a un hogar requieren conexión. Se realizan validaciones de formato correcto en los campos de input.

## Flujo 2: rutinas y tareas




*Figura: Flujo 2 · rutinas y tareas*

Las rutinas se marcan desde el widget sin abrir la app, y cada marca registra quién y cuándo. Las tareas con aviso programan una alarma local en el dispositivo del responsable, que suena aunque no haya conexión.

### Flujo 3: compra, gasto y pago (caso de uso principal)

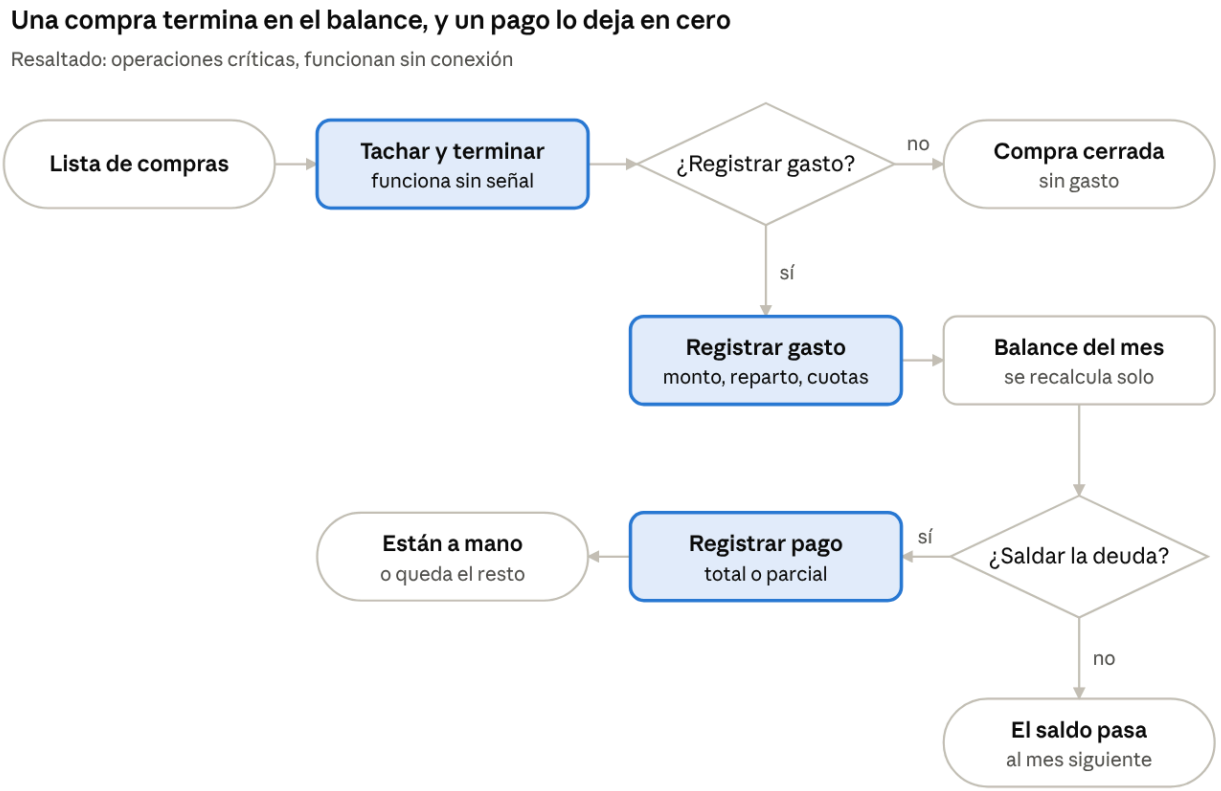



*Figura: Flujo 3 · compra, gasto y pago (caso de uso principal)*

Al terminar una compra, la app ofrece registrar el gasto y el balance se recalcula solo. Si el pago es parcial, el balance muestra lo que falta; si no se salda, la deuda pasa al mes siguiente.

## 13. Arquitectura

La app sigue MVVM con tres capas de Clean Architecture (presentación, dominio y datos) y un enfoque offline first en el que Room es la única fuente de verdad. La UI solo conoce al ViewModel, el ViewModel solo conoce casos de uso, y el dominio define interfaces que la capa de datos implementa. Cada decisión de esta sección se justifica a partir del problema y de los requisitos de las secciones 9 y 10.

### Qué le exige el problema a la arquitectura

Antes de elegir patrones identificamos las condiciones del problema que más restringen el diseño:

| Condición | De dónde surge | Qué le exige a la arquitectura |
|---|---|---|
| Uso sin conexión | Supermercado sin señal y transporte (sección 4); RNF01, RNF02. | Leer y escribir no puede depender de la red. |
| Respuesta inmediata | RNF04, RNF05. | Ninguna pantalla espera a la red para mostrarse. |
| Varios puntos de entrada | App, widget y notificación (RF02, RF03). | La misma regla (por ejemplo, marcar una rutina) se ejecuta igual desde los tres, aun con la app cerrada. |

| Condición | De dónde surge | Qué le exige a la arquitectura |
|---|---|---|
| Reglas de negocio no triviales | Balance con cuotas, pagos parciales y saldo arrastrado; reinicio diario de rutinas (RF04, RNF11). | Las reglas se prueban aisladas, sin emulador. |
| Dos dispositivos que editan los mismos datos | Hogar de dos (sección 5). | Sincronización y resolución de conflictos definidas y bajo nuestro control. |
| Dependencia de un proveedor externo | Sección 15: Supabase, con límites de plan gratuito. | Cambiar el proveedor remoto no debe afectar al dominio ni a la UI. |
| Equipo de dos con áreas separadas | Sección 17. | Límites claros entre UI y datos para trabajar en paralelo. |

## Flujo de datos

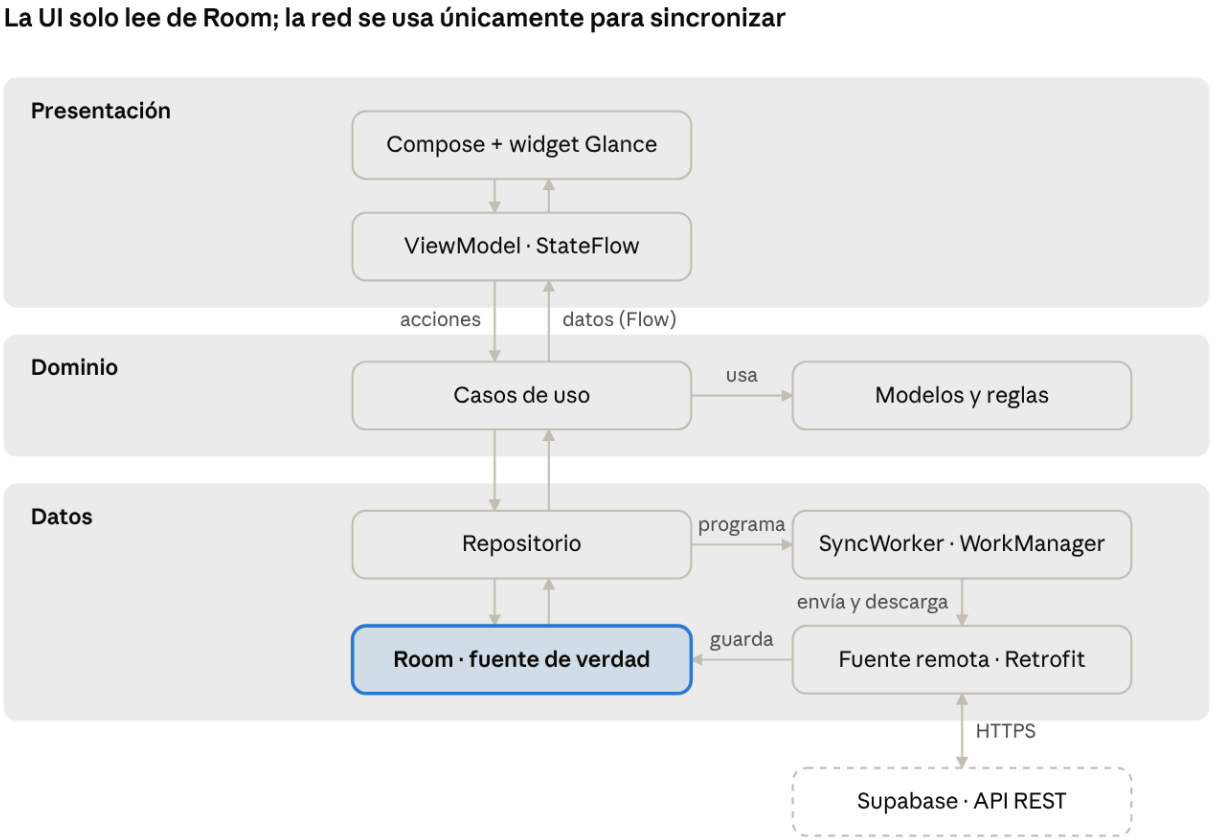



*Figura: Flujo de datos · 3 capas, Room como fuente de verdad*

Las acciones bajan de la UI a Room; los datos suben siempre desde Room como Flow. La red nunca alimenta la UI directamente: SyncWorker descarga cambios, los guarda en Room y Room notifica a la pantalla.

| Capa | Responsabilidad | Componentes |
|---|---|---|
| Presentación | Mostrar el estado y capturar acciones. Sin reglas de negocio. | Pantallas Compose, widget con Glance, un ViewModel por pantalla, UiState (carga, contenido, vacío, error, offline). |
| Dominio | Reglas del hogar, sin dependencias de Android. | Modelos (Rutina, Tarea, Compra, Gasto, Pago, Integrante), casos de uso (MarcarRutina, RegistrarTarea, CompletarTarea, CompletarCompra, RegistrarGasto, RegistrarPago, ObtenerBalanceDelMes), interfaces de repositorio. |

| Capa | Responsabilidad | Componentes |
|---|---|---|
| Datos | Guardar, leer y sincronizar. | Repositorios, DAOs de Room, API Retrofit, mappers entre entidad, modelo y DTO, SyncWorker, programador de avisos. |

**Ejemplo de recorrido:** Tomás completa una compra. La pantalla llama a `onCompletar()` del ViewModel, que ejecuta `CompletarCompra`. El repositorio guarda el cambio en Room con syncState "pendiente" y programa SyncWorker. Room emite el nuevo Flow, el ViewModel actualiza el estado y la lista se redibuja al instante. Cuando hay conexión, SyncWorker envía el cambio a Supabase.

## Topología de infraestructura

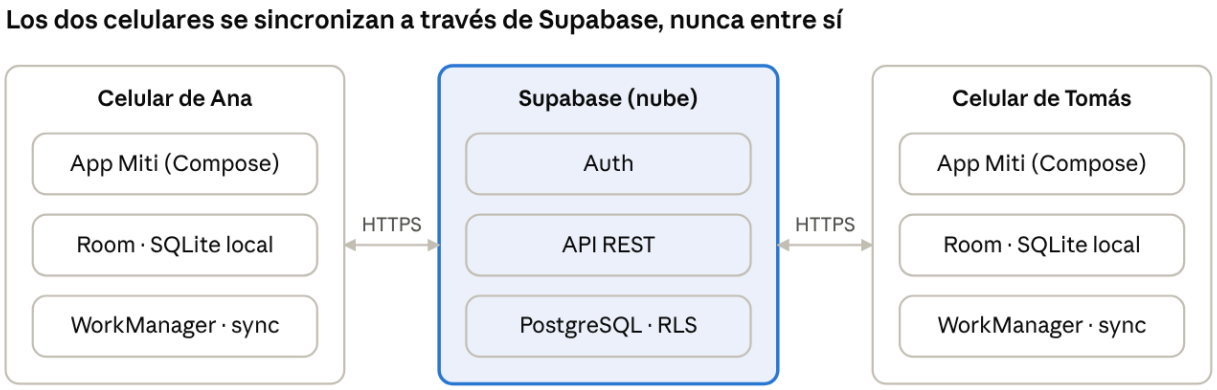



*Figura: Topología · 2 dispositivos y 1 backend*

Cada celular tiene su propia base Room y se comunica solo con Supabase por HTTPS. Supabase Auth identifica a cada integrante y las reglas de acceso por fila (RLS) de PostgreSQL impiden leer datos de otros hogares.

## Decisiones de arquitectura y su justificación

Cada decisión indica qué alternativas se evaluaron, por qué se eligió esta y qué costo aceptamos al hacerlo.

### D01 · MVVM en la capa de presentación

- **Alternativas evaluadas:** MVP y MVI.
- **Por qué:** Compose es declarativo: cada pantalla es una función de su estado. MVVM encaja directamente con ese modelo: el ViewModel expone un único `StateFlow<UiState>` (carga, contenido, vacío, error, offline, los mismos estados diseñados en la sección 11) y la pantalla solo lo dibuja. El ViewModel sobrevive a las rotaciones y es el patrón recomendado por Android y por la materia. MVP obliga al presenter a llamar a la vista, lo que choca con la UI declarativa. MVI agrega reductores y eventos tipados que, para pantallas de listas y formularios como las de Miti, suman código sin un beneficio claro.
- **Costo aceptado:** un ViewModel por pantalla. El widget y las notificaciones no tienen ViewModel; se resuelve con D03.

### D02 · Tres capas con un dominio en Kotlin puro

- **Alternativa evaluada:** dos capas, con el ViewModel hablando directamente con el repositorio o los DAOs, algo habitual en apps chicas.
- **Por qué:** las reglas más delicadas de Miti (cuota del mes, pagos parciales, saldo pendiente que pasa al mes siguiente, reinicio diario de rutinas) son justamente las que RNF11 pide verificar. Al vivir en un dominio sin dependencias de Android se prueban con JUnit en milisegundos y sin emulador

(RNF13). Además, separa con claridad el trabajo de cada integrante: la UI consume casos de uso y la capa de datos implementa interfaces, así ambos pueden avanzar en paralelo acordando solo los contratos del dominio.

- **Costo aceptado:** más clases (modelos de dominio y mappers entre entidad, modelo y DTO) y algunos casos de uso triviales que solo delegan en el repositorio. Los aceptamos a cambio de una regla uniforme ("el ViewModel solo habla con casos de uso") que es fácil de revisar en cada PR.

### D03 · Los mismos casos de uso para la app, el widget y la notificación

- **Alternativa evaluada:** que el widget y la acción de la notificación escriban directamente en Room.
- **Por qué:** marcar una rutina desde el widget o completar una tarea desde la notificación ocurre con la app cerrada y sin ViewModel. Si cada punto de entrada escribiera por su cuenta, la lógica se duplicaría y podría divergir (por ejemplo, olvidar registrar quién marcó o dejar el cambio sin `syncState` pendiente, con lo que nunca se sincronizaría). En cambio, la acción del widget y el receptor de la notificación obtienen por Hilt el mismo caso de uso (`MarcarRutina`, `CompletarTarea`) que usa la app. Así el resultado es idéntico venga de donde venga (RF03, RNF05).
- **Costo aceptado:** configurar puntos de entrada de Hilt fuera de las Activities (widget, receptor y SyncWorker).

### D04 · Room como única fuente de verdad y flujo de datos unidireccional

- **Alternativas evaluadas:** leer de la red y usar la base local solo como caché; usar la caché offline propia de Firestore (ver sección 15).
- **Por qué:** con una sola fuente, la UI nunca decide entre dato local y remoto, y se comporta igual con o sin conexión (RNF01). Las lecturas son locales, lo que permite mostrar Hoy en menos de un segundo (RNF04). La app y el widget observan las mismas tablas, así que no pueden mostrar cosas distintas (RNF05). Como Room emite Flow, cuando cambia un gasto el balance se recalcula y se redibuja sin ninguna acción extra (RF04). Las acciones solo bajan y los datos solo suben, lo que hace que el origen de cada cambio en pantalla sea fácil de rastrear.
- **Costo aceptado:** lo que se ve puede estar desactualizado respecto del otro celular hasta la próxima sincronización. Se mitiga mostrando la hora de la última sincronización (RF02) y sincronizando al abrir la app.

### D05 · Repositorios definidos como interfaces en el dominio

- **Alternativa evaluada:** que los casos de uso usen directamente los DAOs y el cliente Retrofit.
- **Por qué:** por inversión de dependencias, el dominio no sabe que existen Room ni Supabase. Esto tiene dos ventajas concretas para este proyecto: en los tests se reemplazan los repositorios por fakes en memoria, y, si en el futuro hubiera que reemplazar Supabase (por costos o por los límites del plan gratuito), el cambio solo afecta a la capa de datos.
- **Costo aceptado:** una interfaz y una implementación por cada grupo de datos (rutinas, pendientes, gastos y pagos, hogar).

### D06 · Sincronización con WorkManager y cambios pendientes marcados en Room

- **Alternativas evaluadas:** sincronizar desde una corrutina del ViewModel; un servicio en primer plano; una conexión en tiempo real (Supabase Realtime).
- **Por qué:** una corrutina muere cuando se cierra la app, y un cambio hecho en el supermercado podría no enviarse nunca. Un servicio en primer plano exige una notificación permanente, que es invasiva para algo que no es urgente. El tiempo real solo funciona con la app abierta y con conexión, y lo que hay que resolver es justamente qué pasa cuando no la hay. WorkManager ejecuta el trabajo cuando hay red aunque la app esté cerrada o el teléfono se haya reiniciado (RNF02) y reintenta con espera creciente (RNF09). Cada cambio local queda marcado como pendiente en Room, y un único SyncWorker primero envía lo pendiente y después descarga lo nuevo.

- **Costo aceptado:** el trabajo periódico tiene un mínimo de 15 minutos, así que con la app cerrada un cambio del otro integrante puede tardar ese tiempo en llegar. Para un hogar no hace falta tiempo real; además se sincroniza al abrir la app y después de cada cambio.

### D07 · Identificadores UUID generados en el dispositivo y borrado lógico

- **Alternativa evaluada:** ids numéricos asignados por el servidor.
- **Por qué:** sin conexión no hay servidor que asigne ids. Con ids del servidor habría que usar ids temporales y después reescribir las relaciones (por ejemplo, la compra que apunta a su gasto). Un UUID generado en el celular es definitivo desde que se crea. El borrado lógico (`deleted`) permite que un borrado viaje como un cambio más y llegue al otro dispositivo; un borrado físico no dejaría rastro que sincronizar.
- **Costo aceptado:** claves más largas y filas borradas que se acumulan, irrelevante con el volumen de un hogar.

### D08 · Conflictos: marcas que solo se agregan y "gana la última modificación" para el resto

- **Alternativas evaluadas:** combinar cambios campo por campo, estructuras sin conflictos (CRDT) o pedirle al usuario que elija.
- **Por qué:** con dos personas, los conflictos reales son raros, y las alternativas agregan una complejidad desproporcionada. El caso más frecuente de edición simultánea (los dos marcan la misma rutina) se elimina desde el modelo: cada marca es un registro nuevo que nunca se edita. Para tareas, compras y gastos, gana el `updatedAt` más reciente, una regla determinista y fácil de probar (detalle en la sección 14).
- **Costo aceptado:** si los dos editan el mismo gasto sin conexión, una edición se pierde sin aviso. Además, la regla depende del reloj de cada celular; con la hora automática de la red el desvío es de segundos, irrelevante a esta escala.

### D09 · Los datos derivados se calculan, no se guardan

- **Alternativa evaluada:** guardar el balance acumulado y el progreso de cada rutina y actualizarlos con cada cambio.
- **Por qué:** un acumulado guardado en dos dispositivos no se puede reconciliar con "gana la última modificación": si cada uno carga un gasto sin conexión, uno de los dos totales pisaría al otro y el balance quedaría mal para siempre. Si el balance se calcula a partir de los gastos y pagos sincronizados, siempre coincide con el cálculo manual (RNF11). Lo mismo con el progreso de una rutina: el reinicio diario no necesita ningún proceso, porque un registro de ayer simplemente no cuenta hoy.
- **Costo aceptado:** recalcular en cada lectura. Con el volumen de un hogar es trivial, y RNF04 se mide con 200 ítems cargados.

### D10 · Avisos con alarmas locales y no con push desde un servidor

- **Alternativa evaluada:** notificaciones push con Firebase Cloud Messaging disparadas desde funciones en el servidor.
- **Por qué:** la alarma se programa en el celular del responsable y suena a la hora fijada aunque no haya conexión (RNF06). No requiere escribir ni mantener código en el servidor. Las alarmas se vuelven a programar al reiniciar el teléfono, ya que Android las borra al apagarse.
- **Costo aceptado:** el aviso de "tenés una tarea nueva asignada" llega recién cuando el celular del responsable sincroniza, no en el momento en que se crea. En Android 12 o superior, las alarmas exactas requieren un permiso que el usuario puede rechazar; en ese caso se usa una alarma aproximada, no se puede garantizar el desvío máximo de RNF06 y se informa en Hoy, igual que con el permiso de notificaciones.

### D11 · Un backend central; los celulares nunca se comunican entre sí

- **Alternativas evaluadas:** sincronización directa entre celulares (Bluetooth o Wi-Fi local) y un backend propio (por ejemplo, Ktor o Spring).
- **Por qué:** la sincronización directa exige que los dos celulares estén cerca, justo cuando menos se necesita coordinar. Un backend propio implica programar, alojar y asegurar un servidor, lo que excede el alcance y los tiempos del proyecto. Un servicio administrado como Supabase da autenticación, una API REST consumible con Retrofit y aislamiento de hogares por fila (RNF10) sin código de servidor. Al ser un único punto central, cada dispositivo solo sincroniza contra él, y no entre sí.
- **Costo aceptado:** dependencia de un proveedor externo y de los límites del plan gratuito. Se mitiga con D05: el proveedor queda encerrado en la capa de datos.

## Trazabilidad entre requisitos y decisiones

| Requisito | Decisiones que lo resuelven |
|---|---|
| RF01 · Registrar un ítem | D04, D06, D07, D10 |
| RF02 · Consultar lo de hoy | D04, D09 |
| RF03 · Completar un ítem | D03, D04, D08 |
| RF04 · Gastos, pagos y balance | D02, D04, D09 |
| RNF01 · Offline | D04, D06, D07 |
| RNF02 y RNF09 · Confiabilidad y reintentos | D06 |
| RNF04 y RNF05 · Rendimiento y consistencia widget–app | D03, D04 |
| RNF06 · Puntualidad de avisos | D10 |
| RNF10 · Privacidad | D11 |
| RNF11 y RNF13 · Consistencia y mantenibilidad | D02, D05, D09 |

# 14. Persistencia y estrategia Offline First

Room es la única fuente de verdad de la app: la UI y el widget siempre leen de la base local, y la red solo sirve para sincronizar. Toda la información es generada por los usuarios; no se consumen APIs externas de datos.

## Entidades principales

| Entidad | Campos principales | Relaciones |
|---|---|---|
| Hogar | id, nombre, código de unión | Tiene 1 o 2 integrantes y todos los ítems. |
| Integrante | id, hogarId, nombre, email | Pertenece a un hogar. Es responsable de tareas y pagador de gastos. |
| Rutina | id, hogarId, título, frecuencia (diaria / días de la semana), días, vecesPorDía (1 a 3) | Tiene muchos registros. |
| RegistroRutina | id, rutinaId, integranteId, fechaHora | Cada marca de una rutina. Las marcas de hoy definen su progreso. |
| Pendiente | id, hogarId, tipo (tarea / compra), título, responsableId, fechaHora (opcional), avisar, resuelto, gastoId (opcional) | Una compra resuelta puede vincularse a un gasto. |

| Entidad | Campos principales | Relaciones |
|---|---|---|
| Gasto | id, hogarId, descripción, montoTotal, pagadorId, reparto (50/50 / 100/0), beneficiarioId (si es 100/0), cuotasTotales, cuotaInicial, mesInicio | Puede originarse en una compra. |
| Pago | id, hogarId, deIntegranteId, aIntegranteId, monto, fecha | Lo registra quien saldó la deuda. Se descuenta del balance del mes de su fecha. |

Todas las entidades sincronizables llevan además: **id tipo UUID** generado en el dispositivo (para crear sin conexión sin chocar ids), **updatedAt**, **deleted** (borrado lógico) y **syncState** (sincronizado / pendiente de enviar).

Tres datos **no se persisten** porque se calculan, así nunca quedan desactualizados: el balance (a partir de los gastos y los pagos, arrastrando el saldo de los meses anteriores), el progreso de una rutina (a partir de los registros de hoy) y su reinicio diario (un registro de ayer simplemente no cuenta hoy).

DataStore guarda preferencias simples: sesión, hogar activo, onboarding visto y fecha de la última sincronización.

## Comportamiento según la conectividad

| Situación | Qué pasa | Qué ve el usuario |
|---|---|---|
| Tiene conexión | Los cambios se guardan en Room y se envían al remoto enseguida. Se descargan cambios del otro integrante al abrir la app y periódicamente. | Todo actualizado; indicador de última sincronización. |
| Pierde conexión | Los cambios se guardan en Room con syncState "pendiente". La app y el widget siguen siendo usables por completo, y los avisos programados suenan igual porque son locales. No se puede ingresar ni crear o unirse a un hogar. | Banner "Sin conexión"; los ítems no enviados muestran un ícono de pendiente de sincronizar. |
| Recupera conexión | WorkManager detecta la red, envía los cambios pendientes y descarga los del remoto posteriores a la última sincronización. | El banner desaparece y los íconos de pendiente se van. |
| Información local desactualizada | Se muestra igual, con la hora de la última sincronización, mientras se actualiza en segundo plano. | Datos locales y "última sincronización hh:mm". |
| Sin información local todavía | Primer uso o reinstalación: se necesita conexión para ingresar y descargar el hogar. | Estado de carga; sin conexión, mensaje que pide conectarse. |

## Fuente remota y actualización

- **Fuente remota:** Supabase: autenticación con email y contraseña, y base de datos accedida por una API REST con Retrofit.
- **Cuándo se actualiza:** al abrir la app, al guardar un cambio con conexión, al recuperar la red y cada 15 minutos en segundo plano (mínimo de WorkManager).
- **Avisos:** al guardar o sincronizar una tarea con aviso, el dispositivo de su responsable programa una alarma local para esa hora. Además, si una descarga trae una tarea nueva asignada al usuario, se muestra una notificación.
- **Widget:** se actualiza cada vez que cambian las rutinas en Room, y al pasar la medianoche para reiniciar el día.

### Errores y conflictos de sincronización

- **Marcas de rutina:** los registros solo se agregan, nunca se editan, así que no hay conflicto. Si los dos marcan la misma rutina sin conexión, quedan ambos registros y la rutina figura completa.
- **Conflicto en tareas, compras y gastos** (los dos editaron el mismo ítem sin conexión): gana la modificación con **updatedAt** más reciente. Es simple y razonable porque en un hogar de dos es raro editar el mismo ítem a la vez.
- **Borrado vs. edición:** el borrado lógico viaja como un cambio más; si el borrado es más reciente, el ítem desaparece en ambos dispositivos y se cancela su aviso.
- **Error de red o del servidor:** el cambio queda pendiente y WorkManager reintenta con espera creciente. El usuario nunca pierde lo cargado.
- **Límite del hogar:** unirse a un hogar que ya tiene dos integrantes se rechaza en el servidor con un mensaje claro.

## 15. Tecnologías previstas

Se usa Kotlin con Jetpack Compose, el stack para el que está pensada la materia. Cada tecnología responde a una necesidad concreta del problema.

| Tecnología | Para qué se usa | Por qué |
|---|---|---|
| Kotlin + Android | Lenguaje y plataforma. | Requisito de la consigna; soporte nativo de coroutines. |
| Jetpack Compose + Material 3 | UI declarativa con componentes de Android. | Pantallas que reaccionan al estado (carga, vacío, error, offline) y componentes consistentes con el sistema. |
| Jetpack Glance | Widget de pantalla de inicio. | Permite escribir el widget con la misma lógica de Compose y leer de Room (RF02, RF03). |
| Navigation Compose | Navegación entre pantallas y barra inferior. | Flujo tipado y centralizado, con back stack predecible. |
| ViewModel | Estado de cada pantalla. | Sobrevive rotaciones y separa la UI de la lógica (MVVM). |
| Coroutines + Flow / StateFlow | Operaciones asíncronas y flujo de datos reactivo. | Room emite Flow: cuando cambia un gasto o una rutina, el balance y el widget se actualizan solos. |
| Room | Base local, única fuente de verdad. | Datos estructurados con relaciones (rutina ↔ registros, compra ↔ gasto) y funcionamiento offline completo (RNF01). |
| DataStore | Sesión, hogar activo, onboarding visto, última sincronización. | Datos clave-valor que no justifican una tabla. |
| Retrofit + kotlinx.serialization | Comunicación con la API REST remota. | Estándar de la materia para networking; tipado y fácil de testear con fakes. |
| WorkManager | Sincronización en segundo plano y reintentos. | Garantiza el envío al volver la conexión aunque la app esté cerrada (RNF02, RNF09). |
| AlarmManager + notificaciones locales | Avisos de tareas programadas y de tareas asignadas. | Suenan a la hora fijada sin conexión ni servidor de push (RNF06). |
| Supabase (Auth + Postgres + API REST) | Cuentas de usuario y fuente remota. | Autenticación con email lista para usar, API REST consumible con Retrofit y reglas por fila para aislar hogares (RNF10). Plan gratuito. |
| Hilt | Inyección de dependencias. | Permite cambiar fuentes reales por fakes en tests y respeta la separación en capas. |

| Tecnología | Para qué se usa | Por qué |
|---|---|---|
| JUnit + Turbine | Tests unitarios de casos de uso y Flows. | Verificar el balance, el reinicio diario de rutinas y la sincronización (RNF11). |

**Por qué no Firestore directo:** su caché offline propia haría redundante a Room y dejaría la lógica de sincronización fuera de nuestro control, justo lo que la materia pide diseñar y defender.

# 16. Repositorio y forma de trabajo

- **URL del repositorio:** github.com/rociobottinelli/miti
- **Integrantes:** Rocío Bottinelli y Juan Cruz Bautista Serra (ver sección 17).

**Ramas:**

- `main`: siempre compila y corre; solo recibe merges desde `develop` al cerrar cada entrega.
- `develop`: integración continua del equipo.
- `feature/<tema>` (por ejemplo `feature/balance`, `feature/sync`): una rama por funcionalidad, creada desde `develop`.

**Commits:** pequeños y frecuentes, en español y en imperativo, con prefijo de tipo: `feat:`, `fix:`, `docs:`, `test:`, `refactor:`. Ejemplo: `feat: calcular cuota del mes en el balance`.

**Pull requests:** toda rama entra a `develop` por PR, con al menos una revisión de otro integrante, descripción de qué cambia y cómo se probó. No se hacen commits directos a `main` ni a `develop`.

**Organización:** las tareas se registran como issues de GitHub vinculados a los PR, para que la evolución del trabajo quede visible en el repositorio.

# 17. Integrantes, roles y responsabilidades

El equipo tiene dos integrantes. Cada uno lidera un área, comparten la arquitectura y el dominio, y ambos deben poder explicar el funcionamiento completo de la app.

| Integrante | Rol principal | Responsabilidades | Áreas compartidas |
|---|---|---|---|
| Rocío Bottinelli | UI, UX y presentación | Diseño en Figma, pantallas en Compose, widget con Glance, estados de pantalla, accesibilidad, notificaciones en la UI. | Arquitectura, casos de uso, testing, documentación. |
| Juan Cruz Bautista Serra | Datos y sincronización | Room, repositorios, API remota con Retrofit, autenticación, WorkManager, resolución de conflictos, programación de avisos. | Arquitectura, casos de uso, testing, documentación. |

**Trabajo compartido:** los casos de uso del dominio (balance, cuotas, reinicio de rutinas) se reparten por funcionalidad, y cada PR lo revisa el otro integrante, así ambos conocen todo el código.